

Food Store

Sistema de Gestión de Pedidos de Comida
Consola + Colecciones en Memoria

Materia: Programación 2

Alumno: Gustavo Campestre

Institución: Universidad Tecnológica Nacional (UTN)

Carrera: Tecnicatura Universitaria en Programación

Fecha de entrega: 16 de Junio 2025

■ Repositorio GitHub:

https://github.com/ghernanc/Trabajo_practico_integrador_Programacion2

■ Video demostrativo:

<https://www.youtube.com/watch?v=JvGN0OzYxjl&t=51s>

Índice

1-Marco teórico	Página 2.
2-Decisiones técnicas y de arquitectura.....	Página 3.
3-Dificultades técnicas y soluciones	Página 18.
4-Bibliografía.....	Página 19.

1. Marco Teórico

Programación Orientada a Objetos (POO)

La POO organiza el software en torno a objetos que combinan estado (atributos) y comportamiento (métodos). Los cuatro pilares aplicados en este proyecto son:

- **Encapsulamiento:** todos los atributos de las entidades son privados y se acceden mediante getters y setters.
- **Herencia:** la clase abstracta Base centraliza los atributos comunes (id, eliminado, createdAt) que todas las entidades heredan.
- **Polimorfismo:** cada entidad sobrescribe toString() y equals() con comportamiento específico.
- **Abstracción:** la interfaz Calculable define el contrato de comportamiento para el cálculo de totales, implementado por Pedido.

Interfaces en Java

Una interfaz define un contrato de comportamiento que las clases están obligadas a implementar. En este proyecto se definió la interfaz `Calculable`, que declara el método abstracto `calcularTotal()` y un método `default` llamado `resumenTotal()`, que provee una implementación base reutilizable sin necesidad de sobrescribirla. La clase `Pedido` implementa esta interfaz, garantizando que el total siempre se calcule de forma consistente cada vez que se agrega o elimina un detalle.

Colecciones de Java

El framework de Colecciones de Java provee estructuras de datos dinámicas. En este proyecto se utiliza `ArrayList` como implementación de la interfaz `List`, que permite almacenar objetos en memoria con acceso indexado y redimensionamiento automático. Se eligió `ArrayList` por su eficiencia en acceso secuencial y búsqueda lineal, que es el patrón de uso predominante en el sistema.

Enumeraciones (Enums)

Los enums de Java permiten definir un conjunto fijo de constantes con nombre, garantizando que solo se puedan asignar valores válidos a un atributo. En este proyecto se utilizan tres enums: `Rol` (ADMIN, USUARIO), `Estado` (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO) y `FormaPago` (TARJETA, TRANSFERENCIA, EFECTIVO). Cada uno incluye un método estático `fromIndex()` para convertir la opción numérica ingresada por el usuario al valor correspondiente,

y `mostrarOpciones()` para listar las opciones en consola de forma reutilizable.

Manejo de Excepciones

Java provee un mecanismo estructurado de manejo de errores mediante bloques `try-catch-finally` y la jerarquía de clases `Exception`. Se crearon tres excepciones propias que extienden `Exception`: `EntidadNoEncontradaException`, `StockInvalidoException` y `ValidacionException`, lo que permite diferenciar semánticamente los distintos tipos de error del dominio y evitar cierres inesperados del programa.

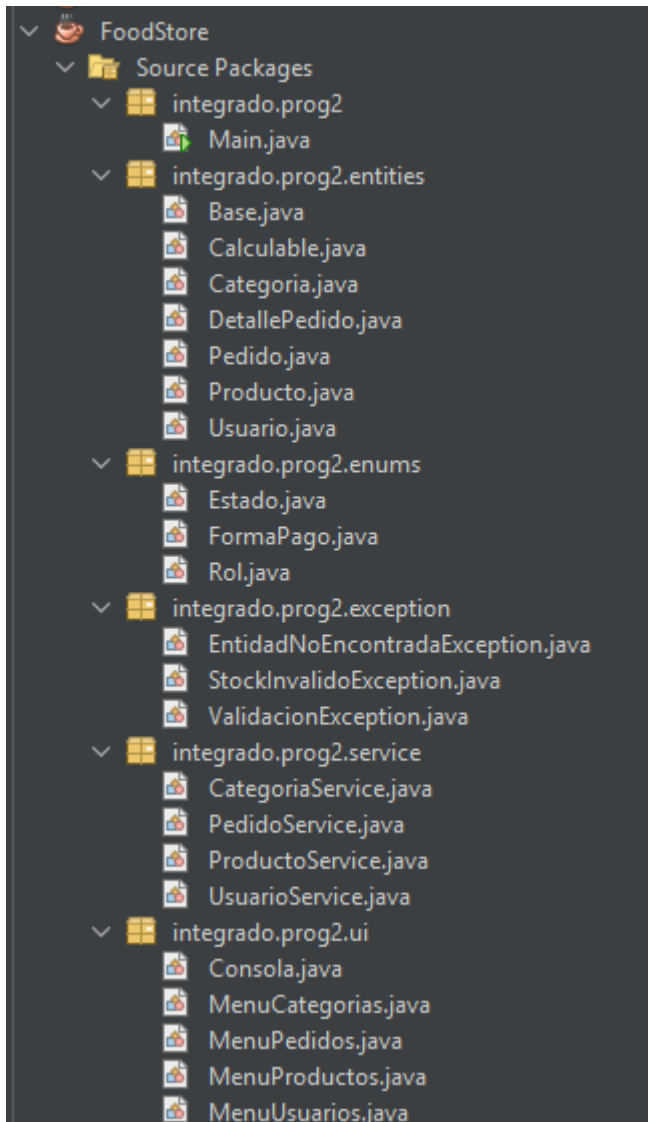
Soft Delete

El soft delete o baja lógica es una técnica que consiste en marcar un registro como eliminado mediante un atributo booleano (`eliminado = true`), en lugar de eliminarlo físicamente de la estructura de datos. Esto permite conservar el historial completo de la información y mantener la integridad referencial: por ejemplo, los pedidos de un usuario eliminado siguen siendo consultables, y los detalles de un pedido eliminado conservan su referencia al producto original.

2. Decisiones Técnicas y Arquitectura

El proyecto se organizó en cinco capas bien diferenciadas, siguiendo el patrón de separación de responsabilidades:

Estructura de paquetes



Capa de Entidades (entities)

Contiene el modelo de dominio puro: clases con atributos, constructores, getters/setters, toString() y equals(). La lógica de negocio intrínseca al objeto (como calcularTotal() en Pedido o addDetallePedido()) también reside aquí, ya que es comportamiento inherente a la entidad.

Capa de Servicios (service)

Cada service gestiona su colección interna (ArrayList) y expone métodos CRUD con validaciones y reglas de negocio. Los IDs se generan con AtomicLong para garantizar unicidad. Los services lanzan excepciones propias que la capa UI captura y muestra al usuario.

Capa de UI (ui)

Contiene los menús de consola. Cada menú recibe sus services por constructor (inyección de dependencias simple) y se encarga únicamente de leer entradas, llamar al service correspondiente y mostrar resultados o errores. La clase Consola centraliza la lectura segura de enteros, Long y Double con reintentos ante entradas inválidas.

Capa de Enumeraciones (enums)

Centraliza los tipos enumerados del dominio: `Rol` (ADMIN, USUARIO), `Estado` (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO) y `FormaPago` (TARJETA, TRANSFERENCIA, EFECTIVO). Cada enum incluye un método estático `fromIndex()` para convertir la opción numérica ingresada por el usuario al valor correspondiente, y `mostrarOpciones()` para listar las opciones en consola de forma reutilizable.

Capa de Excepciones (exception)

Contiene las tres excepciones propias del sistema, todas extendiendo `Exception`: `EntidadNoEncontradaException` para cuando se busca un ID inexistente o eliminado, `StockInvalidoException` para errores relacionados con cantidad o stock insuficiente al crear un detalle de pedido, y `ValidacionException` para errores de validación de campos como mail duplicado, nombre vacío o precio negativo. Separar las excepciones en su propia capa permite identificar semánticamente el tipo de error sin depender de mensajes de texto.

Decisiones de diseño relevantes

- **Soft delete:** se optó por marcar eliminado=true en vez de remover objetos, preservando el historial e integridad referencial.
- **AtomicLong para IDs:** garantiza IDs únicos y crecientes sin necesidad de base de datos.
- **Calculable en Pedido:** el total se recalcula automáticamente cada vez que se agrega o elimina un detalle, asegurando consistencia.
- **Pedido en memoria hasta confirmar:** el pedido no se agrega a la colección hasta completarse exitosamente, evitando estados inconsistentes.

A continuación se muestran los flujos principales del sistema tal como se visualizan en la consola durante la ejecución:

Menú principal

```
=====
  SISTEMA DE PEDIDOS (FOOD STORE)
=====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:
```

Creación de Categoría ,listar y edición,eliminación .

```
--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: pizzas
Descripcion: artesanales
[OK] Categoria creada con ID: 1
--- Operacion finalizada ---

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

=== CATEGORIAS ===
[ID: 1] pizzas           | artesanales

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 3

=== CATEGORIAS ===
[ID: 1] pizzas           | artesanales
ID a editar: 1
Nuevo nombre (Enter para no cambiar) tortas
Nueva descripcion (Enter para no cambiar):
[OK] Categoria actualizada.

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

=== CATEGORIAS ===
[ID: 1] tortas           | artesanales
```

```
--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 4

=== CATEGORIAS ===
[ID: 1] tortas          | artesanales
ID a eliminar: 1
Confirmar eliminacion de 'tortas' (S/N) S
[OK] Categoria eliminada (baja logica).

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
No hay categorias cargadas.

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: |
```

Creación de Producto ,listar y edición,eliminación .


```

=====
SISTEMA DE PEDIDOS (FOOD STORE)
=====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 2

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoría
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: 3
Categorías disponibles:
[ID: 2] pizzas          | artesanales
[ID: 3] bebidas         | jugos
ID de categoría: 2
Nombre: muzzarella
Descripción: grande
Precio: 15000
Stock: 15
Imagen (URL o nombre): muza.jpg
Disponible (S/N): S
[OK] Producto creado con ID: 1

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoría
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: 1

=== PRODUCTOS ===
[ID: 1] muzzarella          | Precio: $15000,00 | Stock: 15 | Disponible: Si | Categoría: pizzas | Descripción: grande

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoría
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione:

```

```

=== PRODUCTOS ===
[ID: 1] muzarella          | Precio: $15000,00 | Stock: 15   | Disponible: Si   | Categoria: pizzas   | Descripcion: grande

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoria
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: 4

=== PRODUCTOS ===
[ID: 1] muzarella          | Precio: $15000,00 | Stock: 15   | Disponible: Si   | Categoria: pizzas   | Descripcion: grande
ID a editar: 1
Editando: [ID: 1] muzarella          | Precio: $15000,00 | Stock: 15   | Disponible: Si   | Categoria: pizzas   | Descripcion: grande
Nuevo nombre (Enter para no cambiar)napolitana
Nueva descripcion (Enter para no cambiar)chica
Nuevo precio (Enter para no cambiar)8000
Nuevo stock (Enter para no cambiar)12
Nueva imagen (Enter para no cambiar)napo.jpg
Disponible S/N (Enter para no cambiar)S
Categorias disponibles:
[ID: 2] pizzas             | artesanales
[ID: 3] bebidas            | jugos
ID de nueva categoria (Enter para no cambiar)2
[OK] Producto actualizado.

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoria
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: 1

=== PRODUCTOS ===
[ID: 1] napolitana         | Precio: $8000,00  | Stock: 12   | Disponible: Si   | Categoria: pizzas   | Descripcion: chica

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoria
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione:

```

```

=== PRODUCTOS ===
[ID: 1] napolitana         | Precio: $8000,00  | Stock: 12   | Disponible: Si   | Categoria: pizzas   | Descripcion: chica

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoria
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: 5

=== PRODUCTOS ===
[ID: 1] napolitana         | Precio: $8000,00  | Stock: 12   | Disponible: Si   | Categoria: pizzas   | Descripcion: chica
ID a eliminar: 1
Confirmar eliminacion de 'napolitana' (S/N)S
[OK] Producto eliminado (baja logica).

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoria
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione: 1
No hay productos cargados.

--- PRODUCTOS ---
1. Listar todos
2. Listar por categoria
3. Crear
4. Editar
5. Eliminar
0. Volver
Seleccione:

```

Creación de Usuario , listar y edición,eliminación .

```
=====
      SISTEMA DE PEDIDOS (FOOD STORE)
=====

1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 3

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: maria
Apellido: lopez
Mail: mlopez@mail.com
Celular: 12345
Contraseña: qwert
Rol:
    1. ADMIN
    2. USUARIO
Seleccione: 1
[OK] Usuario creado con ID: 1

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: pedro
Apellido: sanchez
Mail: psanchez@mail.com
Celular: 12390
Contraseña: uioplk
Rol:
    1. ADMIN
    2. USUARIO
Seleccione: 2
[OK] Usuario creado con ID: 2

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: |
```

```

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

=== USUARIOS ===
[ID: 1] maria      lopez      | Mail: mlopez@mail.com      | Celular: 12345      | Rol: ADMIN
[ID: 2] pedro     sanchez   | Mail: psanchez@mail.com    | Celular: 12390      | Rol: USUARIO

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: |

```

```

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 3

=== USUARIOS ===
[ID: 1] maria      lopez      | Mail: mlopez@mail.com      | Celular: 12345      | Rol: ADMIN
[ID: 2] pedro     sanchez   | Mail: psanchez@mail.com    | Celular: 12390      | Rol: USUARIO
ID a editar: 1
Editando: [ID: 1] maria      lopez      | Mail: mlopez@mail.com      | Celular: 12345      | Rol: ADMIN
Nuevo nombre (Enter para no cambiar) Sofia
Nuevo apellido (Enter para no cambiar) oliva
Nuevo mail (Enter para no cambiar) soliva@mail.com
Nuevo celular (Enter para no cambiar) 09875
Nuevo rol (0 para no cambiar):
  1. ADMIN
  2. USUARIO
Seleccione: 2
[OK] Usuario actualizado.

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

=== USUARIOS ===
[ID: 1] Sofia      oliva      | Mail: soliva@mail.com      | Celular: 09875      | Rol: USUARIO
[ID: 2] pedro     sanchez   | Mail: psanchez@mail.com    | Celular: 12390      | Rol: USUARIO

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: |

```

```

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 4

=== USUARIOS ===
[ID: 1] Sofia          oliva          | Mail: soliva@mail.com          | Celular: 09875          | Rol: USUARIO
[ID: 2] pedro          sánchez          | Mail: psanchez@mail.com        | Celular: 12390          | Rol: USUARIO
ID a eliminar: 1
Confirmar eliminacion de 'Sofia oliva' (S/N) S
[OK] Usuario eliminado (baja logica).

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

=== USUARIOS ===
[ID: 2] pedro          sánchez          | Mail: psanchez@mail.com        | Celular: 12390          | Rol: USUARIO

--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:

```

Creación de Pedido con detalle ,listar y edición,eliminación .

```

=====
      SISTEMA DE PEDIDOS (FOOD STORE)
=====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 4

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 4
Usuarios disponibles:
[ID: 2] pedro          sánchez          | Mail: psanchez@mail.com        | Celular: 12390          | Rol: USUARIO
[ID: 3] roberto        marin            | Mail: rmarin@gmail             | Celular: 1230000        | Rol: USUARIO
ID de usuario: 2
Forma de pago:
  1. TARJETA
  2. TRANSFERENCIA
  3. EFECTIVO
Seleccione: 1

Productos disponibles:
[ID: 2] napolitana      | Precio: $12000,00 | Stock: 12 | Disponible: Si | Categoría: pizzas      | Descripción: chica
[ID: 3] fanta           | Precio: $2000,00  | Stock: 23 | Disponible: Si | Categoría: bebidas     | Descripción: lata
ID de producto (0 para terminar) 2
Cantidad: 1
[OK] Detalle agregado. Subtotal parcial: $12000,00
Agregar otro producto? (S/N) S

Productos disponibles:
[ID: 2] napolitana      | Precio: $12000,00 | Stock: 11 | Disponible: Si | Categoría: pizzas      | Descripción: chica
[ID: 3] fanta           | Precio: $2000,00  | Stock: 23 | Disponible: Si | Categoría: bebidas     | Descripción: lata
ID de producto (0 para terminar) 3
Cantidad: 1
[OK] Detalle agregado. Subtotal parcial: $14000,00
Agregar otro producto? (S/N) N
Total calculado: $14000,00
[OK] Pedido creado con ID: 1 | Total: $14000,00

```

```

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 4
Usuarios disponibles:
[ID: 2] pedro      sanchez      | Mail: psanchez@mail.com      | Celular: 12390      | Rol: USUARIO
[ID: 3] roberto   marin       | Mail: rmarin@gmail          | Celular: 1230000    | Rol: USUARIO
ID de usuario: 3
Forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione: 3

Productos disponibles:
[ID: 2] napolitana      | Precio: $12000,00 | Stock: 11 | Disponible: Si | Categoria: pizzas      | Descripcion: chica
[ID: 3] fanta          | Precio: $2000,00  | Stock: 22 | Disponible: Si | Categoria: bebidas     | Descripcion: lata
ID de producto (0 para terminar) 2
Cantidad: 1
[OK] Detalle agregado. Subtotal parcial: $12000,00
Agregar otro producto? (S/N) N
Total calculado: $12000,00
[OK] Pedido creado con ID: 2 | Total: $12000,00

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione:

```

```

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 1

=== PEDIDOS ===
[ID: 1] Usuario: pedro sanchez      | Estado: PENDIENTE | Pago: TARJETA      | Total: $14000,00 | Fecha: 2026-06-16
[ID: 2] Usuario: roberto marin     | Estado: PENDIENTE | Pago: EFECTIVO     | Total: $12000,00 | Fecha: 2026-06-16

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 2
[ID: 2] pedro      sanchez      | Mail: psanchez@mail.com      | Celular: 12390      | Rol: USUARIO
[ID: 3] roberto   marin       | Mail: rmarin@gmail          | Celular: 1230000    | Rol: USUARIO
ID de usuario: 2
[ID: 1] Usuario: pedro sanchez      | Estado: PENDIENTE | Pago: TARJETA      | Total: $14000,00 | Fecha: 2026-06-16

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 3

=== PEDIDOS ===
[ID: 1] Usuario: pedro sanchez      | Estado: PENDIENTE | Pago: TARJETA      | Total: $14000,00 | Fecha: 2026-06-16
[ID: 2] Usuario: roberto marin     | Estado: PENDIENTE | Pago: EFECTIVO     | Total: $12000,00 | Fecha: 2026-06-16
ID de pedido: 1

[ID: 1] Usuario: pedro sanchez      | Estado: PENDIENTE | Pago: TARJETA      | Total: $14000,00 | Fecha: 2026-06-16
Detalles:
-> Producto: napolitana      | Cantidad: 1      | Subtotal: $12000,00
-> Producto: fanta          | Cantidad: 1      | Subtotal: $2000,00
TOTAL: $14000,00

```

```

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 6

=== PEDIDOS ===
[ID: 1] Usuario: pedro sánchez      | Estado: PENDIENTE   | Pago: TARJETA       | Total: $14000,00 | Fecha: 2026-06-16
[ID: 2] Usuario: roberto marín     | Estado: PENDIENTE   | Pago: EFECTIVO      | Total: $12000,00 | Fecha: 2026-06-16
ID a eliminar: 1
Confirmar eliminacion del pedido ID 1 (S/N): 3
[OK] Pedido eliminado (baja logica).

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 1

=== PEDIDOS ===
[ID: 2] Usuario: roberto marín     | Estado: PENDIENTE   | Pago: EFECTIVO      | Total: $12000,00 | Fecha: 2026-06-16

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: |

```

Error de validación de stock

```

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 4
Usuarios disponibles:
[ID: 2] pedro      sánchez      | Mail: psanchez@mail.com   | Celular: 12390   | Rol: USUARIO
[ID: 3] roberto   marín        | Mail: rmarin@gmail       | Celular: 1230000 | Rol: USUARIO
ID de usuario: 2
Forma de pago:
  1. TARJETA
  2. TRANSFERENCIA
  3. EFECTIVO
Seleccione: 1

Productos disponibles:
[ID: 2] napolitana      | Precio: $12000,00 | Stock: 10 | Disponible: Si | Categoria: pizzas | Descripcion: chica
[ID: 3] fanta          | Precio: $2000,00  | Stock: 22 | Disponible: Si | Categoria: bebidas | Descripcion: lata
ID de producto (0 para terminar): 2
Cantidad: 1000
[!] Stock insuficiente para 'napolitana'. Disponible: 10
[!] El pedido fue cancelado.

--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: |

```

Baja lógica de Categoría con productos activos

```
--- PEDIDOS ---
1. Listar todos
2. Listar por usuario
3. Ver detalles de un pedido
4. Crear pedido
5. Actualizar estado / forma de pago
6. Eliminar pedido
0. Volver
Seleccione: 0

=====
          SISTEMA DE PEDIDOS (FOOD STORE)
=====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 1

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 4

=== CATEGORIAS ===
[ID: 2] pizzas           | artesanales
[ID: 3] bebidas          | jugos
ID a eliminar: 2
[!] La categoría tiene productos activos asociados. No se puede eliminar.

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: |
```


3. Dificultades y Soluciones

Consistencia del pedido ante errores de stock

El principal desafío fue garantizar que un pedido con múltiples detalles no quedara en un estado inconsistente si uno de los productos no tenía stock suficiente. La solución fue construir el Pedido completamente en memoria local (sin agregarlo a la colección del service) y solo persistirlo una vez que todos los detalles se cargaron exitosamente. Ante cualquier `StockInvalidoException`, se abandona el objeto sin guardarlo.

Relación bidireccional Usuario – Pedido

El UML define una relación 1:N entre Usuario y Pedido en ambas direcciones. Implementar esto con colecciones en memoria requirió mantener la consistencia manualmente: al confirmar un pedido, el `PedidoService` lo agrega a su colección, manteniendo ambos lados de la relación actualizados.

Validación de entradas en consola

Leer entradas del usuario por consola es propenso a errores: letras donde se esperan números, Enter vacío, valores negativos. Se centralizó toda la lectura en la clase `Consola` con métodos `leerEntero()`, `leerLong()` y `leerDouble()` que reintentan en loop hasta recibir un valor válido, evitando que el programa se cierre inesperadamente.

Soft delete e integridad referencial

Al eliminar una Categoría que tiene productos asociados, eliminarla lógicamente generaría productos 'huérfanos' activos apuntando a una categoría eliminada. Se decidió impedir la baja de categorías con productos activos, informando el error al operador. Para Pedidos, la eliminación lógica propaga el `eliminado=true` a todos sus `DetallePedido` para mantener la coherencia de la colección.

Manejo del buffer del Scanner

Al combinar `leerEntero()` (que usa `parseInt` sobre `nextLine()`) con lecturas de `String`, se producían lecturas vacías por el salto de línea residual en el buffer. Se resolvió usando `sc.nextLine()` consistentemente en todos los métodos de `Consola`, consumiendo siempre la línea completa.

4. Bibliografía / Webgrafía

Documentación oficial de Java 21: <https://docs.oracle.com/en/java/javase/21/>

Java Collections Framework: <https://docs.oracle.com/javase/tutorial/collections/>

Excepciones en Java: <https://docs.oracle.com/javase/tutorial/essential/exceptions/>

Interfaces en Java:

<https://docs.oracle.com/javase/tutorial/java/landl/createinterface.html>

Enums en Java: <https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>

Universidad Tecnológica Nacional. (2026). *Programación 2* [Material de cursada].
Tecnicatura Universitaria en Programación a Distancia.

